

AERIS CHECKPOINT 8 — ML FOUNDATION (PROMOTED DATA → TRAINING PIPELINE)

Use this as the starting prompt for the next chat.

CONTEXT — WHAT AERIS IS

We are building AERIS, a modular aircraft design and analysis platform.

It is not a collection of scripts.

It is a structured engineering pipeline designed for:

- deterministic geometry generation
- modular sampling strategies
- aerodynamic evaluation (multi-solver capable)
- structured dataset generation
- QC-validated and curated data products
- promotion-controlled datasets
- future ML pipelines
- future dynamics / control / certification workflows

Architecture philosophy:

- modular (generators, samplers, solvers are interchangeable)
 - config-driven
 - reproducible
 - CLI-controlled
 - test-backed
 - production-aware
-

CURRENT STATE — WHAT IS NOW TRUE

Geometry & Sampling

- Geometry generator is modular (registry-ready)
- Sampling (e.g., LHS) works and is decoupled
- Geometry datasets are deterministic and reproducible

Aero System

- Aero solver pipeline works end-to-end
- Supports sweeps across:
 - alpha, beta
 - velocity, altitude
 - angular rates (p , q , r)
 - control inputs
- Solver is modular (currently AVL via AeroSandbox, but replaceable)

Dataset Layer

- Unified aero dataset exists:
 - aero_dataset.csv
 - aero_failures.csv
 - manifest + logs
- Intermediate geometry dataset optionally retained

QC System (Unified & Modular)

- Registry-based validator architecture
- Profiles:
 - basic
 - strict
- Presets:
 - off
 - debug
 - production
 - promotion_strict
- QC integrated in workflow and CLI
- QC results recorded in final_run_summary.json

Curation Layer

- curate-aero command exists
- Produces:
 - curated_aero_dataset.csv
 - rejected_aero_rows.csv
 - curation_report.json
- Rejects:
 - incomplete sweeps
 - failed cases

- non-finite targets
- control diagnostic failures

Promotion Layer (NEW)

- promote-aero command exists
- Writes:
 - promotion_manifest.json
- Encodes:
 - QC context
 - curation outcome
 - promotion decision
 - blockers
- Supports:
 - strict promotion
 - forced promotion (explicit override)

Promotion Gate (NEW)

- require_promoted_aero_dataset(...) exists
 - Enforces:
 - dataset must be promoted
 - curated dataset must exist
 - forced promotion rejected unless explicitly allowed
-

IMPORTANT DESIGN PRINCIPLE

From now on:

Only promoted datasets are allowed for ML or downstream usage

Raw datasets and curated-only datasets are not trusted inputs.

WHERE WE ARE IN THE ROADMAP

We have completed:

- Geometry engine
- Aero engine

- Dataset generation
- QC system
- Curation system
- Promotion system

We are now entering:

ML FOUNDATION PHASE

IMPORTANT DECISION — DYNAMICS IS PAUSED

We intentionally pause:

- mass modeling
- CG estimation
- static margin
- trim
- dynamic stability

Reason:

- ML pipeline requires clean, validated data first
- current datasets are aero-only
- introducing dynamics now would mix concerns prematurely

Dynamics will resume later as:

ML-augmented dynamics / control phase

OBJECTIVE OF THIS PHASE

Build the first ML-ready pipeline using:

promoted aero datasets → training-ready data → baseline models

REQUIREMENTS FOR THIS PHASE

1. Strict Use of Promoted Data

All ML inputs must:

- pass through promotion gate
- originate from curated_aero_dataset.csv
- be validated before use

2. Modular Input Handling

ML pipeline must NOT assume:

- a specific geometry generator
- a specific sampler
- a specific solver (AVL today, others tomorrow)

Instead:

- operate on dataset schema
- not on generator internals

3. Dataset Schema Awareness

ML layer should work with:

- feature columns (design + conditions)
- target columns (aero outputs)

But:

- must validate column existence
- must not assume fixed schema blindly

4. Reproducibility

- deterministic splits (seeded)
 - dataset fingerprinting (optional later)
 - explicit feature/target definitions
-

PHASE TASKS

PHASE 8.1 — Training Data Interface (FIRST STEP)

Create:

`src/aeris/dataset/training_data.py`

Responsibilities:

- load promoted dataset
- validate feature columns
- validate target columns
- handle missing / non-finite values
- return:
 - dataframe
 - X
 - y
 - metadata

CLI:

`aeris dataset training-data --dataset ...`

PHASE 8.2 — Dataset Splitting

Implement:

- train / validation / test split
- grouped split by geometry_id (CRITICAL)

Why:

avoid leakage across same geometry

Support:

- random split
 - grouped split
-

PHASE 8.3 — Baseline ML Models

Implement first simple models:

- Linear regression (baseline sanity)
- Random Forest

- Gradient Boosting (optional)

Targets:

- CL
- CD
- CM

Goal:

verify dataset is learnable

PHASE 8.4 — Training Pipeline

Create:

`src/aeris/ml/train.py`

Capabilities:

- load promoted dataset
 - build X/y
 - split dataset
 - train model
 - evaluate:
 - RMSE
 - MAE
 - R²
 - save:
 - model
 - metrics
 - config snapshot
-

PHASE 8.5 — CLI Integration

Add:

`aeris ml train ...`

Options:

- dataset path
- feature columns
- target columns
- model type

- random seed
-

NON-GOALS (IMPORTANT)

Do NOT:

- implement neural operators yet
- implement GNNs yet
- implement CFD-level surrogates
- integrate dynamics yet
- overcomplicate ML pipeline

This phase is about:

clean, reliable tabular surrogate baseline

SUCCESS CRITERIA

We succeed if:

- promoted dataset loads cleanly
 - X/y built correctly
 - no data leakage
 - models train without failure
 - metrics make physical sense
 - pipeline is reproducible
 - architecture remains modular
-

FINAL NOTE

This is the transition from:

“AERIS generates data”

to:

“AERIS learns from data”

Do not rush complexity.

Build:

- clean
 - controlled
 - extensible
-

START HERE

Begin with:

PHASE 8.1 — training_data module

Provide:

- full code
- tests
- CLI command

Then proceed step-by-step.